

Programação em memória distribuída

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Implementar uma troca de mensagens com exatamente dois processos MPI: o processo 0 envia uma mensagem ao processo 1, que responde com a mesma mensagem. Medir com `MPI_Wtime` o tempo de múltiplas trocas consecutivas para tamanhos de 8 bytes a pelo menos 1 MB. Representar graficamente o tempo em função do tamanho e identificar as regiões dominadas pela latência e pelo custo de transferir os bytes.

2 Fundamentação

No MPI, cada processo possui seu próprio espaço de endereçamento. As rotinas relevantes para este experimento são:

2.1 Inicialização e finalização

- `MPI_Init`: inicializa o ambiente MPI antes das demais chamadas.
- `MPI_Finalize`: encerra o ambiente MPI após a comunicação.

2.2 Identificação dos processos

- `MPI_Comm_rank`: informa o rank do processo em `MPI_COMM_WORLD`; aqui, 0 ou 1.
- `MPI_Comm_size`: informa quantos processos há nesse comunicador; aqui, exatamente 2.

2.3 Comunicação ponto a ponto

- `MPI_Send`: envia uma mensagem no modo padrão.
- `MPI_Recv`: recebe a mensagem correspondente ao envio.

No ping-pong, o rank 0 envia e depois recebe; o rank 1 recebe e responde com o mesmo buffer. Essa ordem evita deadlock.

2.4 Outros modos de envio

- `MPI_Bsend`: envia usando um buffer fornecido pela aplicação com `MPI_Buffer_attach`.
- `MPI_Ssend`: envio síncrono; só conclui depois que o recebimento correspondente começa.

O programa não usa esses dois modos.

3 Experimento

O job 2109294 foi executado na partição intel-512 do NPAD, em um nó identificado no log como r1i3n15. O Slurm reservou duas tarefas, uma CPU por tarefa. O programa foi compilado no job com `mpicc -std=c11 -O3 -Wall -Wextra` e iniciado com `mpirun -np 2`. O estado final foi COMPLETED, código 0:0, em cinco segundos.

Para cada tamanho, ambos os processos alocam e preenchem um buffer. Uma MPI_Barrier os sincroniza antes da medida. O processo 0 mede, com MPI_Wtime, 10 000 pares consecutivos de MPI_Send/MPI_Recv. Alocação, preenchimento, barreira, impressão e finalização do MPI ficam fora do intervalo. O tempo médio é o tempo total dividido por 10 000 e corresponde à ida e volta completa, não a uma comunicação em apenas uma direção. Houve uma execução por tamanho; não foram medidas repetições independentes nem dispersão.

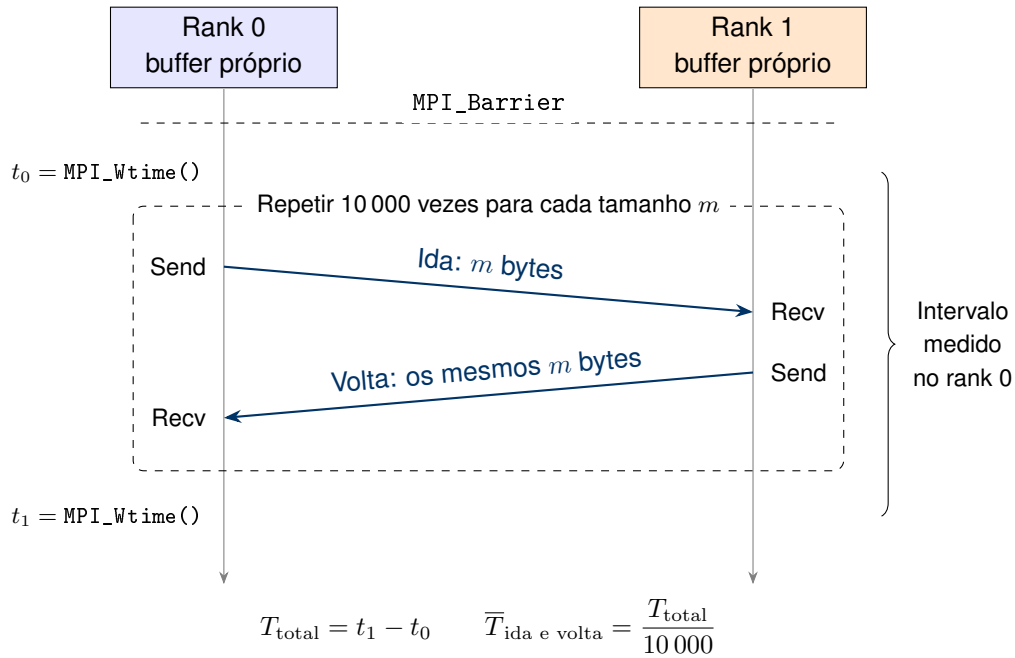


Figura 1: Sequência do ping-pong para um tamanho de mensagem. O tempo avança para baixo, sem escala; as setas indicam transferências, não a duração das chamadas. Os dois ranks executaram no mesmo nó, com espaços de endereçamento separados.

A Figura 1 mostra por que a ordem das chamadas permite a troca: o primeiro envio do rank 0 encontra um recebimento no rank 1; na resposta, os papéis se invertem. O rank 1 reutiliza seu buffer após receber a mensagem. Assim, o protocolo não exige que o MPI armazene temporariamente dois envios sem recebimentos correspondentes.

Cada repetição transfere m bytes na ida e m bytes na volta. Portanto, o tempo médio apresentado na tabela e no gráfico corresponde ao par completo, incluindo os custos das chamadas MPI. A barreira fica antes de t_0 ; o cronômetro do rank 0 termina após receber a última resposta. O experimento não mede separadamente o tempo de uma única direção.

4 Resultados

A Tabela 1 reproduz os valores do arquivo `tarefa14-2109294.out`. A Figura 2 representa o tempo médio por troca; como o número de trocas é constante, o tempo total tem a mesma tendência.

Mensagem (bytes)	Tempo total (s)	Tempo médio por troca (μ s)
8	0,006102265	0,610
64	0,006119762	0,612
512	0,008196224	0,820
4 096	0,048486192	4,849
32 768	0,120328724	12,033
262 144	0,811817241	81,182
1 048 576	3,030861879	303,086

Tabela 1: Medições do job 2109294, com 10 000 trocas por tamanho.

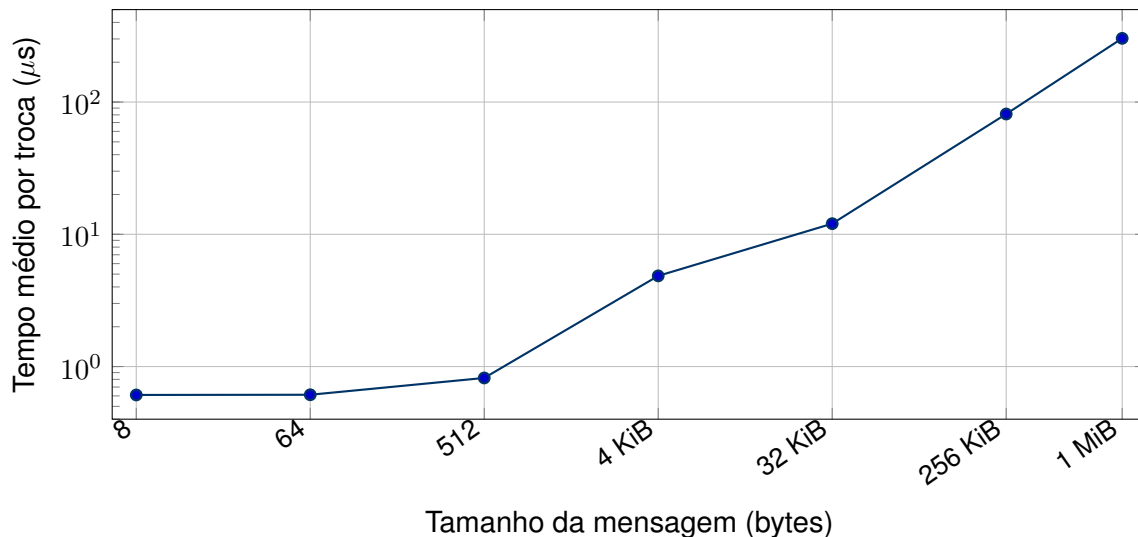


Figura 2: Tempo médio de uma ida e volta em função do tamanho. Ambos os eixos usam escala logarítmica para mostrar os sete pontos.

Entre 8 e 64 bytes, o tempo médio praticamente não muda: é a região em que a latência domina nesta medição. Em 512 bytes já há aumento, mas os sete pontos não localizam precisamente a transição. De 32 KiB a 1 MiB, o tempo cresce com o tamanho da mensagem, indicando maior influência da transferência dos bytes. Como os dois processos foram alocados no mesmo nó, estes dados não demonstram a largura de banda de uma rede entre nós.

Anexo A: Código completo

```
1 #include <mpi.h>
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6
7 #define EXPECTED_PROCESS_COUNT 2
8 #define SENDER_RANK 0
9 #define RECEIVER_RANK 1
10 #define NUM_ITER 10000
11
12 static const int MESSAGE_SIZES[] = {
13     8, 64, 512, 4096, 32768, 262144, 1048576
14 };
15
16 int main(int argc, char **argv) {
17     int rank;
18     int process_count;
19
20     MPI_Init(&argc, &argv);
21     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
22     MPI_Comm_size(MPI_COMM_WORLD, &process_count);
23
24     if (process_count != EXPECTED_PROCESS_COUNT) {
25         if (rank == SENDER_RANK) {
26             fprintf(stderr, "Este programa requer exatamente 2 processos.\n");
27         }
28         MPI_Finalize();
29         return EXIT_FAILURE;
30     }
31
32     const int size_count = sizeof(MESSAGE_SIZES) / sizeof(MESSAGE_SIZES[0]);
33
34     if (rank == SENDER_RANK) printf("Tamanho(bytes)\tTempo_total(s)\tTempo_medio(us)\n");
35
36     for (int index = 0; index < size_count; ++index) {
37         const int message_size = MESSAGE_SIZES[index];
38         char *buffer = malloc((size_t)message_size);
39         if (buffer == NULL) {
40             fprintf(stderr, "Falha ao alocar %d bytes no processo %d.\n",
41                 message_size, rank);
42             MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
43         }
44         memset(buffer, 'a', (size_t)message_size);
45
46         MPI_Barrier(MPI_COMM_WORLD);
47         const double start = MPI_Wtime();
48
49         for (int iteration = 0; iteration < NUM_ITER; ++iteration) {
50             if (rank == SENDER_RANK) {
```

```

50         MPI_Send(buffer, message_size, MPI_CHAR, RECEIVER_RANK, 0,
51                 MPI_COMM_WORLD);
52         MPI_Recv(buffer, message_size, MPI_CHAR, RECEIVER_RANK, 0,
53                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
54     } else {
55         MPI_Recv(buffer, message_size, MPI_CHAR, SENDER_RANK, 0,
56                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
57         MPI_Send(buffer, message_size, MPI_CHAR, SENDER_RANK, 0,
58                 MPI_COMM_WORLD);
59     }
60 }
61
62     const double total_seconds = MPI_Wtime() - start;
63
64     if (rank == SENDER_RANK) printf("%d\t\t%.9f\t\t%.3f\n", message_size,
65     total_seconds, total_seconds * 1e6 / NUM_ITER);
66
67     free(buffer);
68 }
69
70 MPI_Finalize();
71 return EXIT_SUCCESS;
72 }

```

Anexo B: Script Slurm

```

1  #!/bin/bash
2  #SBATCH --job-name=PINGPONG
3  #SBATCH --output=tarefa14-%j.out
4  #SBATCH --ntasks=2
5  #SBATCH --cpus-per-task=1
6  #SBATCH --time=01:00:00
7  #SBATCH --partition=intel-512
8
9  set -e
10
11 mpicc -std=c11 -O3 -Wall -Wextra -o tarefa14 tarefa14.c
12 mpirun -np 2 ./tarefa14

```